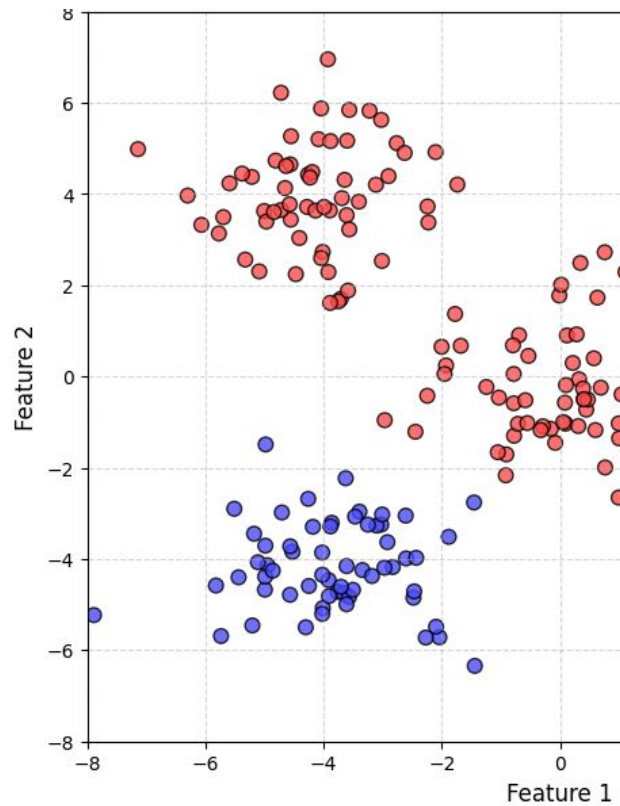


Árboles

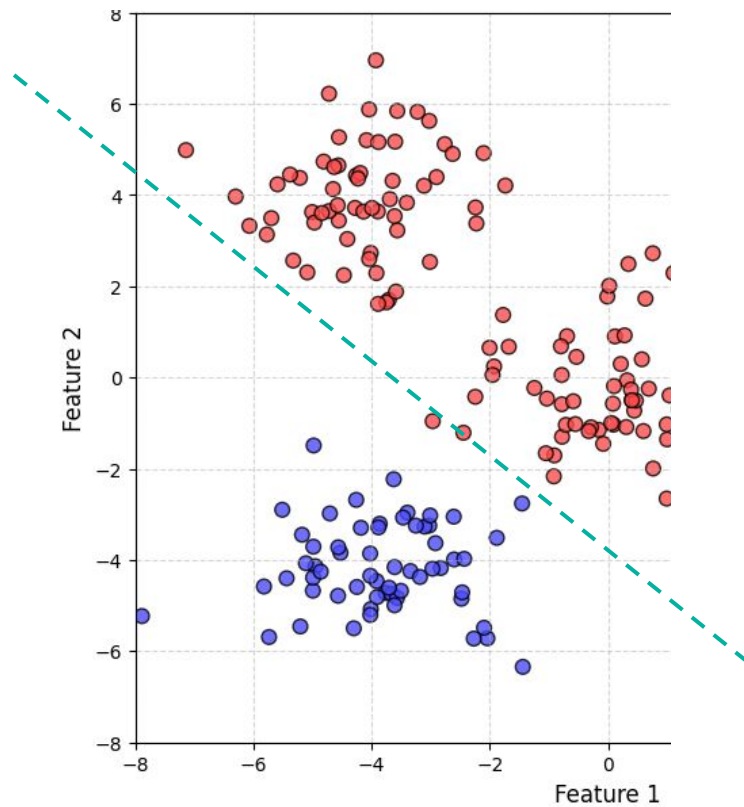
Clase práctica 5

Repaso

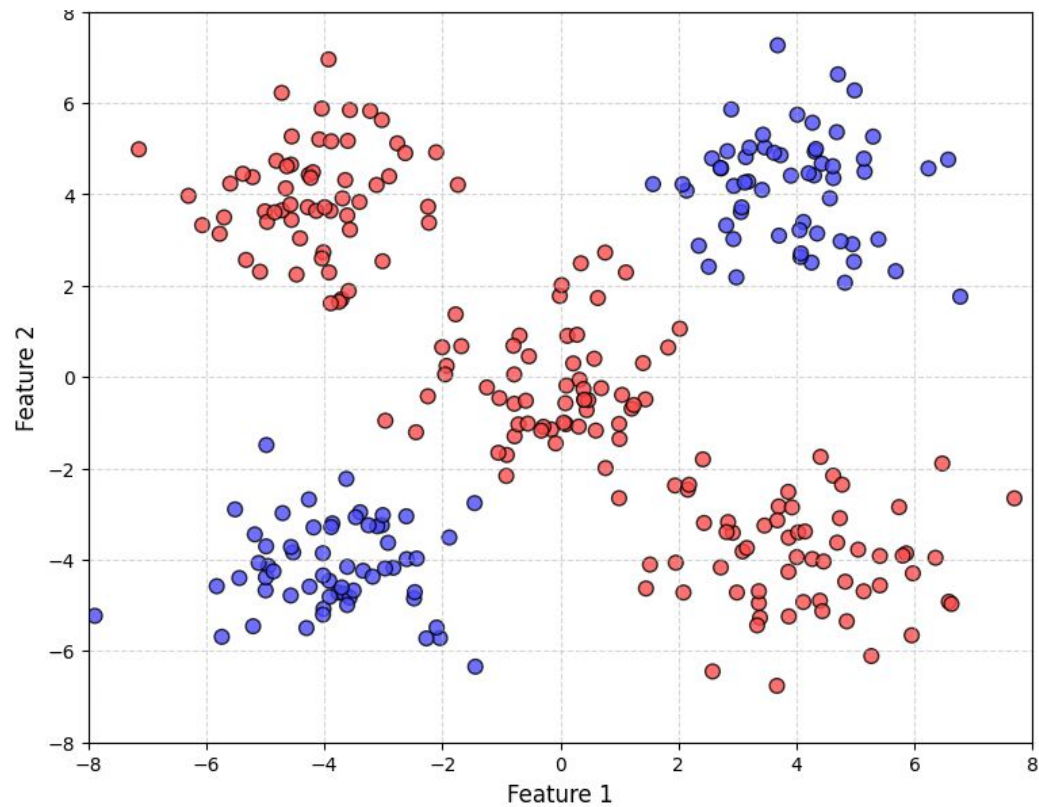
Repaso



Repaso



Repaso



Repaso

Repaso

Un árbol **particiona el espacio de atributos** en regiones

Queremos encontrar las mejores particiones

¿Qué significa **mejor** o **peor**?

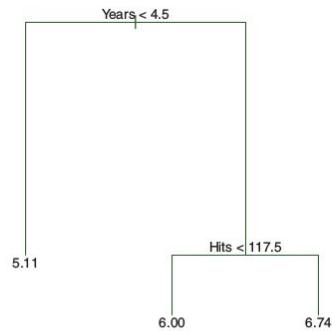
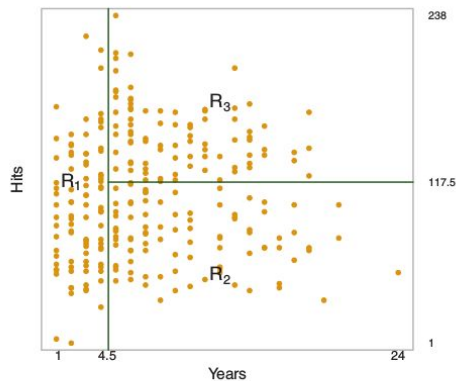


FIGURE 8.2. The three-region partition for the **Hitters** data set from the regression tree illustrated in Figure 8.1.

Árboles - Clasificación

La **impureza de un nodo** nos dice qué tan **mezcladas** están las muestras al mirar su categoría

Árboles - Clasificación

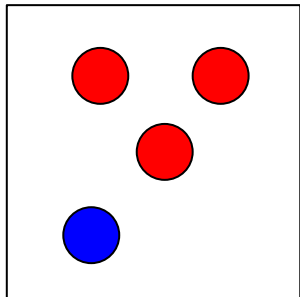
La **impureza de un nodo** nos dice qué tan **mezcladas** están las muestras al mirar su categoría

$$Gini_m = 1 - \sum_{k=1}^C p_{mk}^2$$

Árboles - Clasificación

La **impureza de un nodo** nos dice qué tan **mezcladas** están las muestras al mirar su categoría

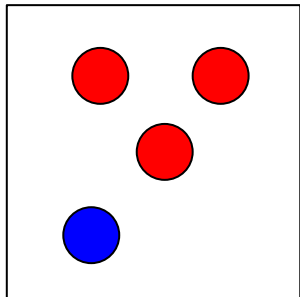
$$Gini_m = 1 - \sum_{k=1}^C p_{mk}^2$$



Árboles - Clasificación

La **impureza de un nodo** nos dice qué tan **mezcladas** están las muestras al mirar su categoría

$$Gini_m = 1 - \sum_{k=1}^C p_{mk}^2$$

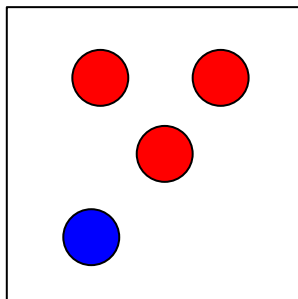


p_rojo = 0.75
p_azul = 0.25

Árboles - Clasificación

La **impureza de un nodo** nos dice qué tan **mezcladas** están las muestras al mirar su categoría

$$Gini_m = 1 - \sum_{k=1}^C p_{mk}^2$$



$$p_{\text{rojo}} = 0.75$$

$$p_{\text{azul}} = 0.25$$

Gini

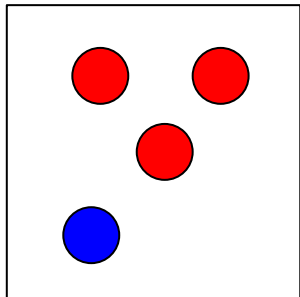
$$1 - (0.75^2 + 0.25^2)$$

$$= \mathbf{0.375}$$

Árboles - Clasificación

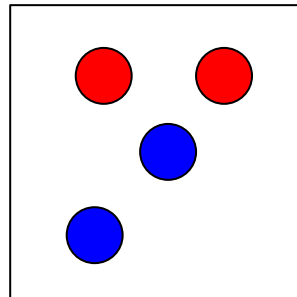
La **impureza de un nodo** nos dice qué tan **mezcladas** están las muestras al mirar su categoría

$$Gini_m = 1 - \sum_{k=1}^C p_{mk}^2$$



$$\begin{aligned} p_{\text{rojo}} &= 0.75 \\ p_{\text{azul}} &= 0.25 \end{aligned}$$

$$\begin{aligned} \text{Gini} \\ 1 - (0.75^2 + 0.25^2) \\ = 0.375 \end{aligned}$$

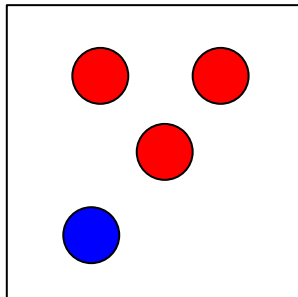


$$\begin{aligned} p_{\text{rojo}} &= 0.50 \\ p_{\text{azul}} &= 0.50 \end{aligned}$$

Árboles - Clasificación

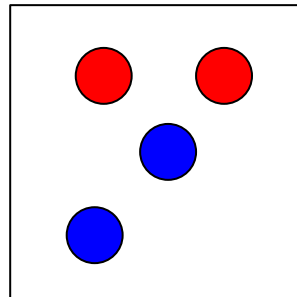
La **impureza de un nodo** nos dice qué tan **mezcladas** están las muestras al mirar su categoría

$$Gini_m = 1 - \sum_{k=1}^C p_{mk}^2$$



$$\begin{aligned} p_{\text{rojo}} &= 0.75 \\ p_{\text{azul}} &= 0.25 \end{aligned}$$

$$\begin{aligned} \text{Gini} \\ 1 - (0.75^2 + 0.25^2) \\ = \mathbf{0.375} \end{aligned}$$



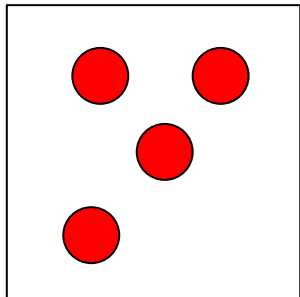
$$\begin{aligned} p_{\text{rojo}} &= 0.50 \\ p_{\text{azul}} &= 0.50 \end{aligned}$$

$$\begin{aligned} \text{Gini} \\ 1 - (0.50^2 + 0.50^2) \\ = \mathbf{0.50} \end{aligned}$$

Árboles - Clasificación

La **impureza de un nodo** nos dice qué tan **mezcladas** están las muestras al mirar su categoría

$$Gini_m = 1 - \sum_{k=1}^C p_{mk}^2$$



$$\begin{aligned} p_{\text{rojo}} &= 1.00 \\ p_{\text{azul}} &= 0.00 \end{aligned}$$

$$\begin{aligned} \textbf{Gini} \\ 1 - (1.00^2 + 0.00^2) \\ = \textbf{0.00} \end{aligned}$$

Árboles - Clasificación

¿Cómo sabemos dónde hacer el **corte**?

¿Y qué **features** utilizamos (y en qué orden)?

Árboles - Clasificación

¿Cómo sabemos dónde hacer el **corte**?

¿Y qué **features** utilizamos (y en qué orden)?

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

[StatQuest](#)

Árboles - Clasificación

¿Cómo sabemos dónde hacer el **corte**?

¿Y qué **features** utilizamos (y en qué orden)?

Probamos con todo!

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

[StatQuest](#)

Árboles - Clasificación

¿Cómo sabemos dónde hacer el **corte**?

¿Y qué **features** utilizamos (y en qué orden)?

Probamos con todo!

Vamos a elegir la **feature** que nos dé el menor:

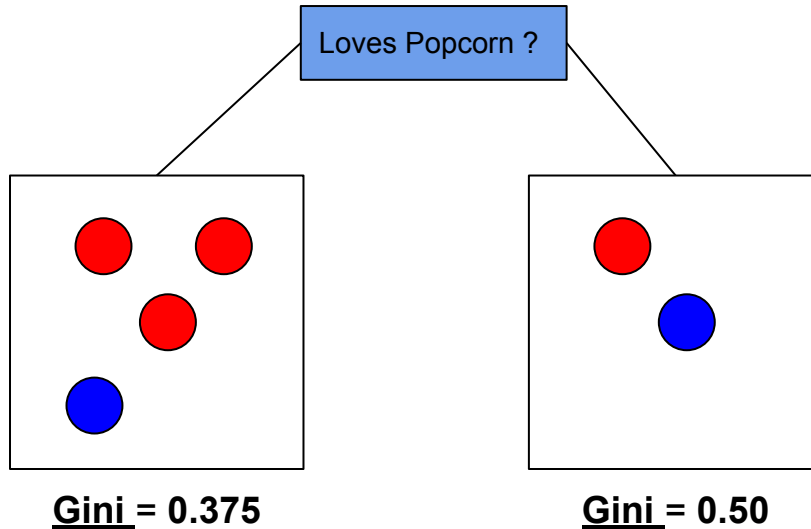
$$Gini_{split} = \frac{N_{left}}{N} Gini_{left} + \frac{N_{right}}{N} Gini_{right}$$

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

[StatQuest](#)

Árboles - Clasificación

$$Gini_{split} = \frac{N_{left}}{N} Gini_{left} + \frac{N_{right}}{N} Gini_{right}$$

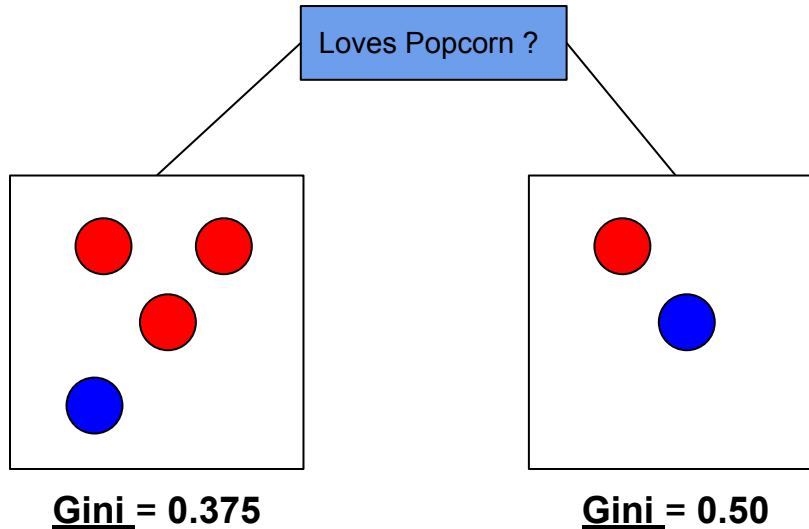


Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

[StatQuest](#)

Árboles - Clasificación

$$Gini_{split} = \frac{N_{left}}{N} Gini_{left} + \frac{N_{right}}{N} Gini_{right}$$



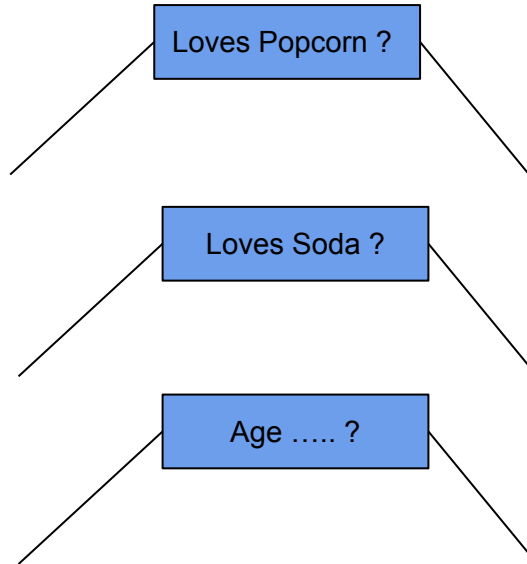
Total Gini : $4/6 * 0.375 + 2/6 * 0.50 = 0.416$

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

[StatQuest](#)

Árboles - Clasificación

$$Gini_{split} = \frac{N_{left}}{N} Gini_{left} + \frac{N_{right}}{N} Gini_{right}$$



Total Gini : 0.416

Total Gini : 0.230

Total Gini : 0.120

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

[StatQuest](#)

Árboles - Clasificación

$$Gini_{split} = \frac{N_{left}}{N} Gini_{left} + \frac{N_{right}}{N} Gini_{right}$$

Lastly, we calculate the **Gini Impurity** values for each average age.

	Age	Loves Cool As Ice	
	7	No	
9.5	12	No	Gini Impurity = 0.429
15	18	Yes	Gini Impurity = 0.343
26.5	35	Yes	Gini Impurity = 0.476
36.5	38	Yes	Gini Impurity = 0.476
44	50	No	Gini Impurity = 0.343
66.5	83	No	Gini Impurity = 0.429

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

[StatQuest](#)

Árboles - Clasificación

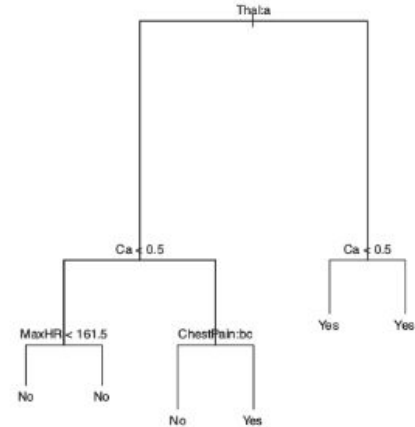
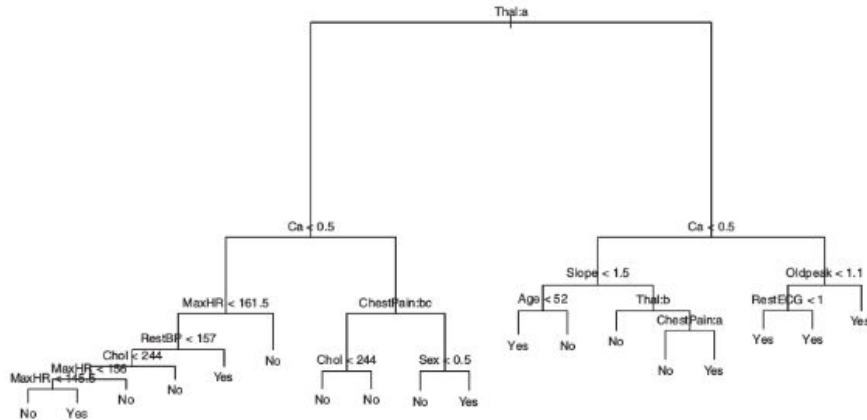
```
para cada feature:  
  para cada threshold:  
    calcular impureza  
  
elegir mejor split  
  
repetir recursivamente
```

Loves Popcorn	Loves Soda	Age	Loves Cool As Ice
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

[StatQuest](#)

Árboles - Clasificación

¿Hasta cuando repetimos esto?



Árboles - Regresión

Cambia la **métrica** para armar nuestro árbol

Árboles - Regresión

Cambia la **métrica** para armar nuestro árbol

$$MSE_{split} = \frac{N_{m_1}}{N_m} MSE_{m_1} + \frac{N_{m_2}}{N_m} MSE_{m_2}$$

Árboles - Regresión

Cambia la **métrica** para armar nuestro árbol

¿Cómo **predecimos** el valor de una muestra nueva?

Árboles - Python

Árboles - Python

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()
model.fit(X, y)

DecisionTreeClassifier()

model.predict(X_test)
```

array([0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1,
1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 1,
1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0,
1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1,
1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1,
0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1])

Árboles - Python

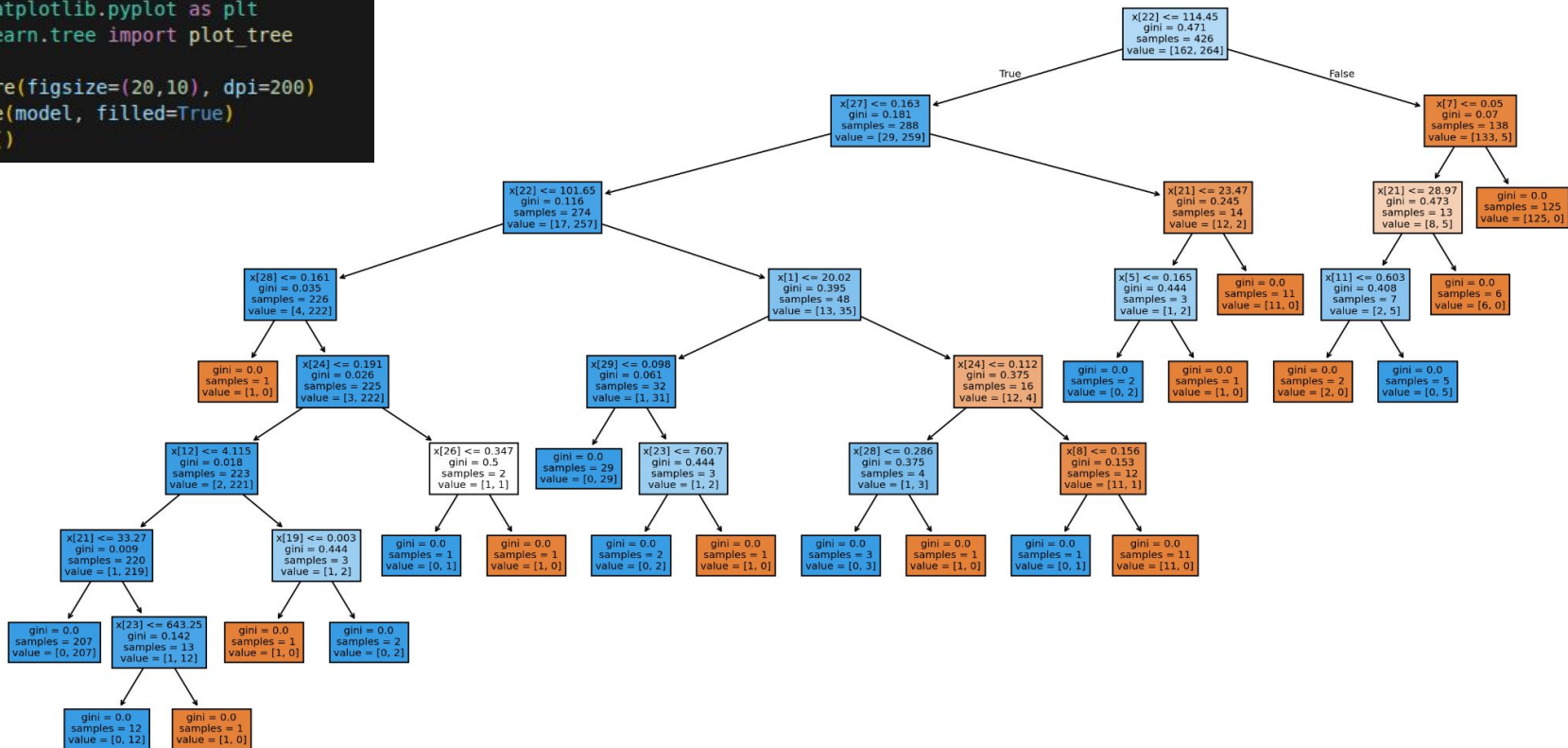
```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(20,10), dpi=200)
plot_tree(model, filled=True)
plt.show()
```

Árboles - Python

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree
```

```
plt.figure(figsize=(20,10), dpi=200)
plot_tree(model, filled=True)
plt.show()
```



Árboles - Python

```
from sklearn.metrics import accuracy_score
```

```
y_pred = model.predict(X_test)  
accuracy_score(y_test, y_pred)
```

✓ 0.0s

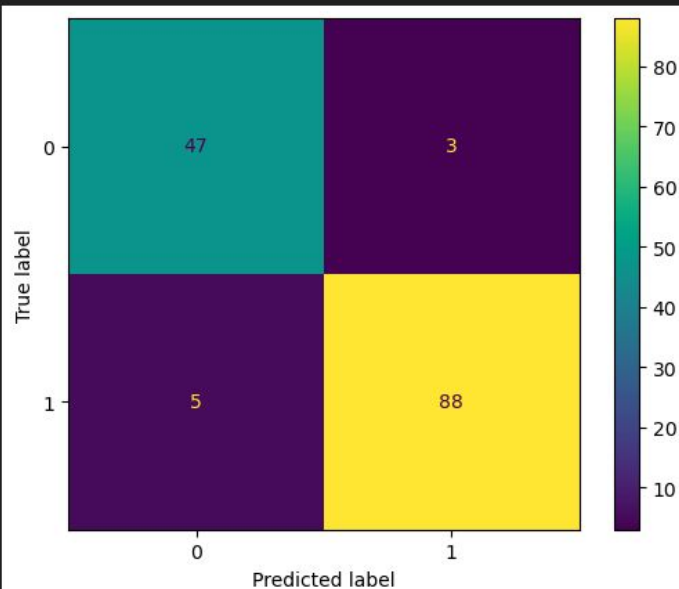
0.9440559440559441

```
from sklearn.metrics import ConfusionMatrixDisplay
```

```
ConfusionMatrixDisplay.from_estimator(model, X_test, y_test)
```

✓ 0.0s

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x771739bf82d0>



Árboles - Python

```
from sklearn.metrics import accuracy_score
```

```
y_pred = model.predict(X_test)  
accuracy_score(y_test, y_pred)
```

✓ 0.0s

0.9440559440559441

```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_test, y_pred))
```

✓ 0.0s

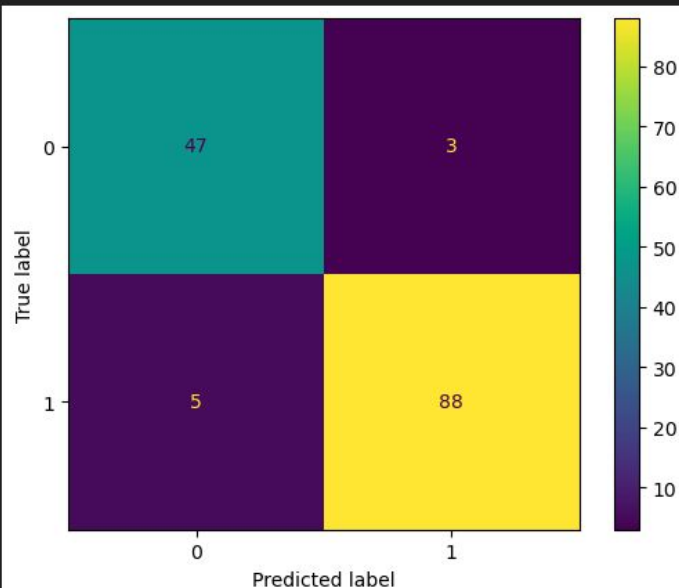
	precision	recall	f1-score	support
0	0.90	0.94	0.92	50
1	0.97	0.95	0.96	93
accuracy			0.94	143
macro avg	0.94	0.94	0.94	143
weighted avg	0.94	0.94	0.94	143

```
from sklearn.metrics import ConfusionMatrixDisplay
```

```
ConfusionMatrixDisplay.from_estimator(model, X_test, y_test)
```

✓ 0.0s

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x771739bf82d0>



Árboles - Python

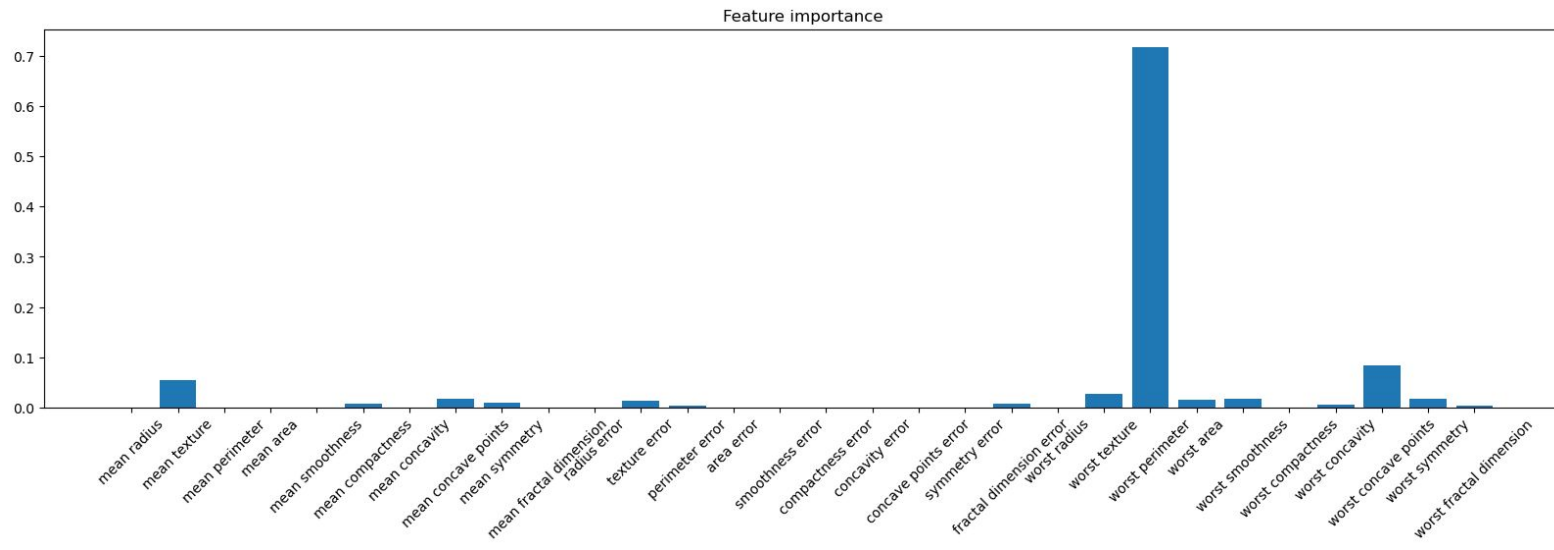
```
import matplotlib.pyplot as plt

plt.figure(figsize=(20,5))

plt.bar(X.columns, model.feature_importances_)
plt.xticks(rotation=45)

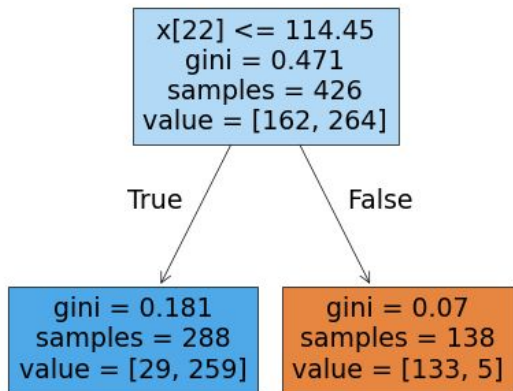
plt.title("Feature importance")
plt.show()
```

Mide cuánto ayudó cada variable a reducir la **impureza**

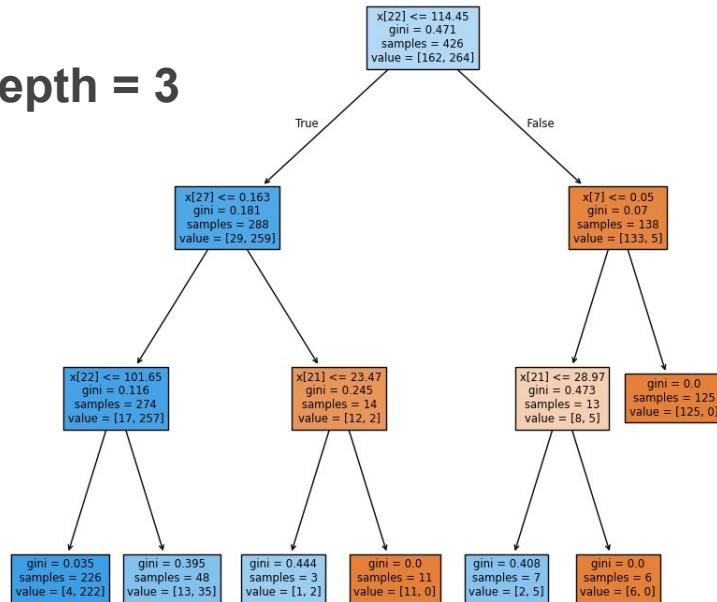


Árboles - Python - Max Depth

```
model = DecisionTreeClassifier(max_depth=1)
model.fit(X, y)
```



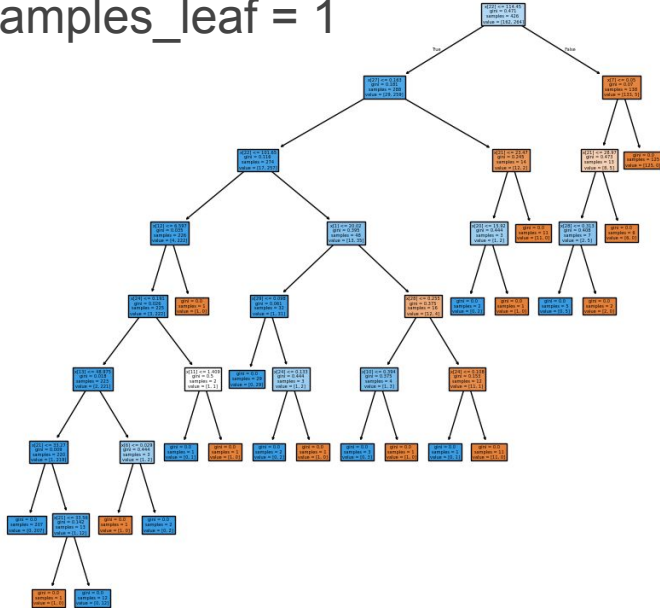
depth = 3



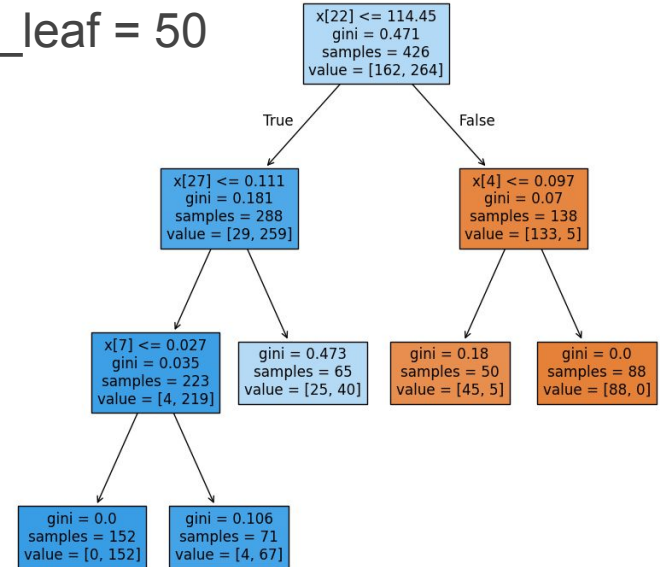
Árboles - Python - Min Samples Leaf

```
model = DecisionTreeClassifier(min_samples_leaf=10)  
model.fit(X, y)
```

min_samples_leaf = 1



min_samples_leaf = 50



Árboles - Python - Otros Hiperparámetros

criterion

→ Define la métrica de impureza que usa el árbol para elegir el mejor split (por ejemplo, *gini* o *entropy*).

max_depth

→ Limita la profundidad máxima del árbol para controlar su complejidad y evitar overfitting.

min_samples_split

→ Es el número mínimo de muestras que debe tener un nodo para poder dividirse en dos.

min_samples_leaf

→ Es el número mínimo de muestras que debe tener una hoja final del árbol.

max_features

→ Limita cuántas variables puede evaluar el modelo en cada split al buscar la mejor división.

ccp_alpha

→ Controla la poda del árbol penalizando su complejidad para reducir overfitting.